HOWARD UNIVERSITY · DEPARTMENT OF MATHEMATICS

Practice Problems: Filtering Rows with WHERE

MATH 153 · Data Wrangling and Data Engineering

Moussa Doumbia, Ph.D.

These problems practice the six families of `WHERE` operators from Lecture 2, Part 3.4, on the Northwind Analytics `employees` table. Work in a new notebook. For every problem, **predict the result before you run the query**, then run it and compare.

The operators

Operator	Meaning	Example
`=, <>, <, >, <=, >=`	comparison	`salary >= 100000`
`AND, OR, NOT`	combine conditions	`department='Sales' AND region='Midwest'`
`BETWEEN a AND b`	inclusive range	`salary BETWEEN 60000 AND 100000`
`IN (...)`	membership	`region IN ('West','Northeast')`
`LIKE`	pattern: `%` = any run, `_` = one char	`phone LIKE '202-%'`
`IS NULL / IS NOT NULL`	missing test	`region IS NULL`

Setup

1. Run the setup code in the Appendix (last page). It builds a fresh database, `northwind_practice.db`, with 14 employees. Two of them are new hires whose `region` has not been assigned yet, so it is stored as `NULL`.
2. Print every result with the `row_factory` pattern from the lecture. Copy this template and change only the SQL, the parameters, and the columns you print:

```
conn.row_factory = sqlite3.Row            # rows become dict-like
c2 = conn.cursor()
c2.execute("""SELECT full_name, position, department, salary
              FROM employees
              WHERE salary >= ?""", (100000,))
for r in c2:
    print(f"   {r['full_name']:<16}{r['position']:<20}"
          f"{r['department']:<14}{r['salary']:>10,.0f}")
conn.row_factory = None                   # back to plain tuples
```

Rules for every problem

- Pass values the user could change (numbers, names, regions) with `?` placeholders, never with f-strings inside the SQL.
- Print the number of rows returned along with the rows themselves.
- When a question asks *why*, answer in one or two sentences in a Markdown cell.

Problem 1. Comparison: =, <>, <, >, <=, >=

(a) List the name, position, department, and salary of everyone who earns **at least** 100,000.

(b) List everyone who is **not** in the Sales department. How many rows did you get?

(c) List the name and `hired_on` date of everyone hired **before** January 1, 2019.

(d) `hired_on` is stored as `TEXT`, not as a date. Explain why `<` still gives the correct answer in part (c). Would it still work if dates were stored as `'03/11/2019'`?

Problem 2. Combining conditions: `AND`, `OR`, `NOT`

(a) List the Sales employees who work in the Midwest region.

(b) List everyone who is in Engineering **or** earns a bonus greater than 20,000. Which person appears only because of the bonus condition?

(c) Count the employees in the West region. Then count the employees for which `NOT region = 'West'` is true. Add the two counts. Do they add up to 14? Explain what happened to the missing rows.

(d) Run these two queries and report the count from each:

```
SELECT COUNT(*) FROM employees
WHERE department = 'Sales' OR department = 'Finance' AND salary > 90000;

SELECT COUNT(*) FROM employees
WHERE (department = 'Sales' OR department = 'Finance') AND salary > 90000;
```

Why are the answers different? Which operator does SQL evaluate first, `AND` or `OR`?

Problem 3. Inclusive range: `BETWEEN a AND b`

(a) List everyone whose salary is between 60,000 and 100,000, sorted from lowest to highest.

(b) Rewrite part (a) without `BETWEEN`, using `>=`, `<=`, and `AND`. Confirm that both versions return the same number of rows.

(c) List the name and `hired_on` date of everyone hired during the calendar years 2020 through 2022. Is an employee hired on `'2022-12-31'` included? What about `'2020-01-01'`?

Problem 4. Membership: `IN (...)`

(a) List everyone in the West or Northeast region using `IN`.

(b) Rewrite part (a) with `OR` instead of `IN`. Which version would you rather maintain if the list grew to six regions?

(c) List everyone whose department is **not** Sales or Engineering, using `NOT IN`.

(d) Does `WHERE region NOT IN ('West')` return the two employees with no region? Run it, then explain the result using what you learned in Problem 2(c).

Problem 5. Pattern matching: `LIKE`

Recall that `%` matches any run of characters (including none), and `_` matches exactly one.

(a) List the name and phone number of everyone in the 202 area code.

(b) List everyone whose position contains the word `Engineer` anywhere. Did `Engineering Mgr` match? Why?

(c) List everyone whose **first** name has exactly four letters. *Hint:* four underscores, then a space.

(d) List everyone whose email has the letter `a` as its **second** character.

(e) The lecture stored `phone` as `TEXT`. Explain in two sentences why part (a) would be hard to write if `phone` had been stored as an `INTEGER` such as `2025550118`.

Problem 6. Missing values: `IS NULL` / `IS NOT NULL`

(a) List the employees whose region is missing.

(b) Run `SELECT ... WHERE region = NULL`. How many rows come back? Explain why, and connect it to `np.nan == np.nan` in Python.

(c) Run `SELECT COUNT(*), COUNT(region) FROM employees`. Why are the two numbers different?

(d) List everyone whose region **is** known, using `IS NOT NULL`.

Problem 7. Putting it together

Write **one** query that uses at least four of the six operator families. It should list the name, department, hire date, phone, and salary of every employee who

- works in Finance or Engineering,
- was hired between January 1, 2018 and December 31, 2022 (inclusive),
- does **not** have a 312 phone number, and
- has a known region,

sorted by salary from highest to lowest. Use `?` placeholders for the two departments, the two dates, and the phone pattern. Print the result with the `row_factory` template.

Stretch. Turn your query into a function `find_staff(depts, start, end)` that accepts a list of departments of any length. *Hint:* build the right number of `?` marks with `", ".join("?" * len(depts))`, and explain why this is still safe from SQL injection.

Submission

- One notebook, exported to PDF, with every query, its output, and your written answers in Markdown cells.
- Problems in order and clearly labeled.

Appendix: setup code

Run this once at the top of your notebook. It deletes any old copy of the table, so you can rerun it at any time to start fresh.

```
import sqlite3

conn = sqlite3.connect("northwind_practice.db")
cur = conn.cursor()
cur.execute("DROP TABLE IF EXISTS employees")
cur.execute('''
CREATE TABLE employees (
    emp_id      INTEGER PRIMARY KEY,
    full_name   TEXT    NOT NULL,
    email       TEXT    UNIQUE,
    phone       TEXT    UNIQUE,
    position    TEXT,
    department  TEXT    DEFAULT 'Unassigned',
    region      TEXT,
    salary      REAL    CHECK (salary > 0),
    bonus       REAL    DEFAULT 0.0 CHECK (bonus >= 0),
    hired_on    TEXT
)''')
staff = [
 (1001,'Amara Johnson','amara.johnson@northwind.com','202-555-0118',
  'Senior Analyst','Finance','Northeast',96500.0,8500.0,'2019-03-11'),
 (1002,'Daniel Osei','daniel.osei@northwind.com','312-555-0142',
  'Sales Rep','Sales','Midwest',61000.0,12400.0,'2021-07-06'),
 (1003,'Fatima Bah','fatima.bah@northwind.com','617-555-0193',
  'Engineering Mgr','Engineering','Northeast',142000.0,21000.0,'2017-01-23'),
 (1004,'Marcus Lee','marcus.lee@northwind.com','415-555-0107',
  'Software Engineer','Engineering','West',118000.0,9000.0,'2022-02-14'),
 (1005,'Zara Nkomo','zara.nkomo@northwind.com','404-555-0164',
  'HR Generalist','HR','South',68000.0,0.0,'2023-05-02'),
 (1006,'Elijah Carter','elijah.carter@northwind.com','312-555-0175',
  'Financial Analyst','Finance','Midwest',79500.0,4200.0,'2020-09-28'),
 (1007,'Priya Raman','priya.raman@northwind.com','206-555-0131',
  'Senior Engineer','Engineering','West',131000.0,15000.0,'2018-11-05'),
 (1008,'Noah Williams','noah.williams@northwind.com','404-555-0149',
  'Operations Assoc','Operations','South',54000.0,0.0,'2024-01-08'),
 (1009,'Grace Mensah','grace.mensah@northwind.com','202-555-0122',
  'Director','Sales','Northeast',155000.0,32000.0,'2016-06-20'),
 (1010,'Omar Haddad','omar.haddad@northwind.com','415-555-0188',
  'Data Analyst','Finance','West',84000.0,3500.0,'2022-10-17'),
 (1011,'Lena Petrova','lena.petrova@northwind.com','617-555-0156',
  'Ops Manager','Operations','Northeast',99000.0,7000.0,'2019-08-12'),
 (1012,'Tomas Rivera','tomas.rivera@northwind.com','312-555-0113',
  'Sales Rep','Sales','Midwest',58500.0,9800.0,'2023-03-27'),
 (1013,'Ines Duarte','ines.duarte@northwind.com','206-555-0177',
  'Data Engineer','Engineering',None,112000.0,6000.0,'2025-09-02'),
 (1014,'Kwame Boateng','kwame.boateng@northwind.com','202-555-0102',
  'Recruiter','HR',None,71000.0,2000.0,'2025-09-15'),
]
cur.executemany("INSERT INTO employees VALUES (?,?,?,?,?,?,?,?,?,?)", staff)
conn.commit()

cur.execute("SELECT COUNT(*) FROM employees")
print("employees loaded:", cur.fetchone()[0])      # should print 14
```